# Doctrine

OVIGNE Loïc - DUT INFO



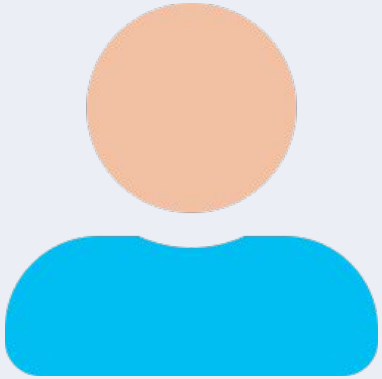
# Base de données

Une base de données permet de stocker des informations propres à une application, par exemple la liste des utilisateurs.

Ces données peuvent être structurées, c'est à dire qu'il y a un modèle relationnel entre les données (mySQL, SQLServer …) ou elles peuvent êtres brutes non structurées (noSQL).

Des requêtes écrites en SQL (Structured Query Language) permettent de lire et écrire dans la base de données.

# Structure d’une base de données



- Chaque type d’information est définie dans une table
- Une colonne correspond à une valeur typée (chaîne, entier etc.)
- Une ligne correspond à une entrée

Exemple de table *Personne*:

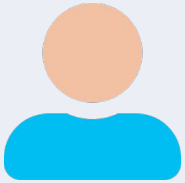
| Nom | Prénom | Naissance | Sexe |
|---|---|---|---|
| DANLMUR | Alphonse | 25/10/1975 | M |
| TATOUILLE | Lara | 06/08/1989 | F |

# Clé primaire

Il peut y avoir deux lignes avec les mêmes informations, pour les identifier l'une de l'autre, il doit y avoir une valeur unique, c'est la clé primaire.



Livre
ISBN



Personne
Num. Sécurité Social

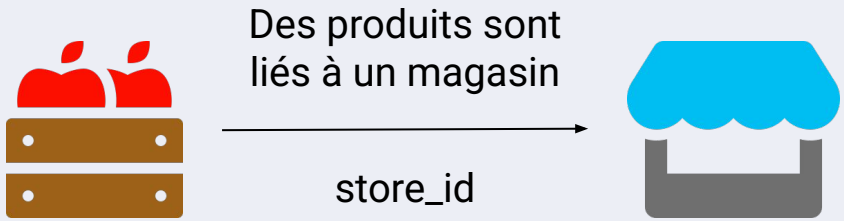


Entreprise
SIREN

Dans la plupart des tables, une colonne **id** sert de clé primaire

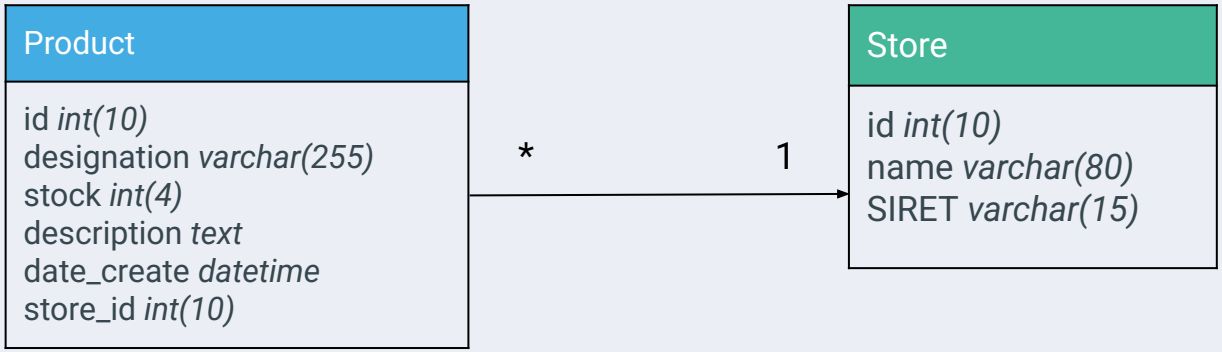
# Relations et clés étrangères

Dans une base de données relationnel, les clés étrangères sont utilisées pour faire des relations



# Modèle de données

Une application repose principalement sur la structure de sa base de données, l'utilisation de modèle conceptuel de données (MCD) permet de visualiser la structure d'une base.



# Doctrine

ORM - Object Relational Mapping
Fait le lien entre la base de données et des objets PHP

```php
class Comment
{
    public string $pseudo;
    public \DateTime $date;
    public string $content;
    public bool $valid;
}
```






Base de données



Requêtes SQL

# Entity

Les entités sont les classes utilisées pour gérer les données
Ces données sont définies grâce aux attributs

```php
namespace App\Entity;

use Doctrine\DBAL\Types\Types;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity]
class Article
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(type: Types::STRING)]
    private string $title = '';

    #[ORM\Column(type: Types::DATETIME_MUTABLE, nullable: true)]
    private ?\DateTime $date;
```

| Name | Type | AI | Null |
|---|---|---|---|
| id(PK) | integer(11) | true | false |
| title | varchar(255) | false | false |
| date | datetime | false | true |

# Migrations

Classes PHP contenant des requêtes SQL pour mettre à jour le schéma de base de données

- Permet d'appliquer la même modification sur tous les environnements
- Donne la possibilité de revenir en arrière avec les requêtes inverses

```php
class Version2
{
    public function up(): void
    {
        $this->addSql('ALTER TABLE user ADD phone VARCHAR(15) DEFAULT NULL');
    }

    public function down(): void
    {
        $this->addSql('ALTER TABLE user DROP phone');
    }
}
```

# Commandes

Pour faciliter la maintenance de la base de données, il existe des commandes très utiles

*doctrine:database:create* - crée la base de données

*make:migration* - crée automatique un fichier de migration avec les requêtes

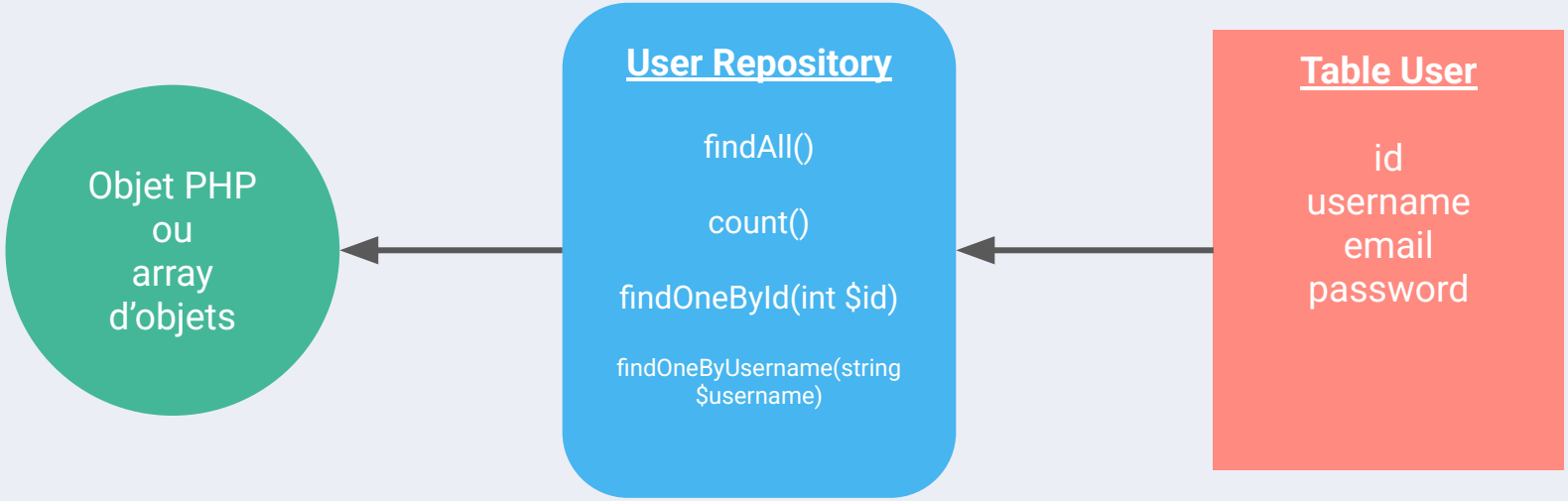
*doctrine:migrations:migrate* - exécute les migrations non exécutées

*doctrine:migrations:list* - affiche les migrations et leurs statuts

*make:entity* - permet de créer une entité

# Repository

Un repository est un objet permettant d'accéder aux données d'une table



# Relations entre entités

```php
namespace App\Entity;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity]
class Article
{
    #[ORM\ManyToOne(inversedBy: 'articles')]
    private ?Category $category = null;
}
```

```php
namespace App\Entity;

use Doctrine\Common\Collections\Collection;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity]
class Category
{
    #[ORM\OneToMany(targetEntity: Article::class, mappedBy: 'category')]
    private Collection $articles;
}
```

